

Detailed Notes

Chapter 7 — Structures, Unions, Enumerations, and typedef

Topics Covered

- Introduction — Five Ways to Create Custom Types
 - Structures
 - Declaration, Tag, and Object
 - Accessing Structure Members (dot operator)
 - Structure Assignments
 - Arrays of Structures
 - Mailing List Example Program
 - Passing Structures to Functions
 - Passing Structure Members to Functions
 - Passing Entire Structures (call by value)
 - Type Matching Requirement
 - Structure Pointers
 - Declaring and Using Structure Pointers
 - The Arrow Operator (->)
 - Software Timer Example Program
 - Arrays and Structures within Structures
 - Array Members
 - Nested Structures
 - Unions
 - Declaration and Memory Sharing
 - Accessing Union Members
 - putw() Example (type conversion using union)
 - Bit-Fields
 - General Form and Types
 - Serial Port Status Byte Example
 - Unnamed Bit-Fields and Mixing with Normal Members
 - Restrictions
 - Enumerations
 - Declaration and Integer Values
 - Initializers and Custom Values
 - I/O of Enumeration Values
 - C vs C++ Difference (tags vs full type names)
 - Using sizeof to Ensure Portability
 - typedef

CHAPTER 7 — STRUCTURES, UNIONS, ENUMERATIONS, AND typedef

The C language gives you five ways to create a custom data type:

Custom Type	Keyword	Description
Structure	Struct	A grouping of variables under one name. Also called an aggregate, compound, or conglomerate data type.
Union	Union	Enables the same piece of memory to be defined as two or more different types of variables simultaneously.
Bit-field	(inside struct/union)	A special type of structure or union element that allows easy access to individual bits within a byte or word.
Enumeration	Enum	A list of named integer constants — provides a way to associate meaningful names with integer values.
Type alias	Typedef	Defines a new name for an existing type. Does NOT create a new physical type.

STRUCTURES

A structure is a collection of variables referenced under one name, providing a convenient means of keeping related information together. A structure declaration forms a template that can be used to create structure objects (instances of a structure). The variables that make up the structure are called members (also called elements or fields).

The keyword `struct` tells the compiler that a structure is being declared:

```
struct addr
{
    char        name[30];
    char        street[40];
    char        city[20];
    char        state[3];
    unsigned long zip;
};
/* Note: terminated by a semicolon — a structure declaration IS a statement */
```

Term	Meaning
Structure tag (<code>addr</code>)	Identifies this particular data structure — serves as its type specifier
Structure declaration	Defines the form (template) of the data only — no variable is created yet
Structure variable (object)	An actual instance in memory — created by a separate declaration
Members	The individual variables inside the structure (name, street, city, state, zip)

Declaring Structure Variables

At the point of declaring a structure, no variable has actually been created — only the form of the data has been defined. To declare a variable of type `addr`:

```
struct addr addr_info;
/* addr describes the FORM (type); addr_info is an INSTANCE (object) */
```

You can also declare one or more objects at the same time as the structure definition:

```
struct addr {
    char        name[30];
    char        street[40];
    char        city[20];
    char        state[3];
    unsigned long zip;
} addr_info, binfo, cinfo;
/* Declares three structure variables of type addr */
```

If only one structure variable is needed, the tag is optional:

```
struct {
    char    name[30];
    char    street[40];
    char    city[20];
    char    state[3];
    unsigned long zip;
} addr_info; /* no tag — declares exactly one unnamed structure variable */
```

General form of a structure declaration:

```
struct tag {
    type member-name;
    type member-name;
    ...
} structure-variables;
/* Either tag or structure-variables may be omitted, but NOT both */
```

NOTE When a structure variable is declared, the compiler automatically allocates sufficient memory to accommodate ALL of its members. Each structure variable contains its OWN COPIES of all members — changes to one variable's member do not affect another variable's member.

Accessing Structure Members — The Dot Operator

Individual members of a structure are accessed through the `.` operator (the dot operator). The general form is:

```
object-name.member-name
```

Examples:

```
addr_info.zip = 12345;          /* assign ZIP code */
printf("%lu", addr_info.zip);  /* print ZIP code */
```

```
gets(addr_info.name);           /* read name string */

/* To access individual characters of an array member, index the ELEMENT: */
for(t=0; addr_info.name[t]; ++t)
    putchar(addr_info.name[t]);
/* Index is placed after the member name (name), NOT after the object name (addr_info)
*/
```

Structure Assignments

The information in one structure can be assigned to another structure of the same type using a single assignment statement — there is no need to assign each member separately:

```
#include <stdio.h>
int main(void)
{
    struct { int a; int b; } x, y;
    x.a = 10;
    y = x;           /* assign entire structure in one statement */
    printf("%d", y.a); /* prints 10 */
    return 0;
}
```

ARRAYS OF STRUCTURES

Structures are often arrayed. To declare an array of structures, first define the structure type, then declare an array variable of that type:

```
struct addr addr_list[100];
/* Creates 100 sets of variables organized as defined in struct addr */
```

- To access a specific structure in the array, index the array name
- To access a specific element within a structure, use the dot operator after the indexed array name
- Like all arrays, arrays of structures begin indexing at 0

```
printf("%lu", addr_list[2].zip); /* print ZIP of the 3rd structure */
addr_list[2].name[0] = 'X';     /* assign to 1st char of name in 3rd structure */
```

Mailing List Example Program

This program illustrates how arrays of structures are used in a practical application. It uses an array of `addr` structures to hold name and address information with four operations: enter, delete, list, and quit.

```
/* A simple mailing list using an array of structures. */
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

struct addr {
    char    name[30];
    char    street[40];
```

```

    char        city[20];
    char        state[3];
    unsigned long zip;
} addr_list[MAX];

void init_list(void), enter(void), delete(void), list(void);
int  menu_select(void), find_free(void);

int main(void)
{
    char choice;
    init_list();
    for(;;) {
        choice = menu_select();
        switch(choice) {
            case 1: enter(); break;
            case 2: delete(); break;
            case 3: list(); break;
            case 4: exit(0);
        }
    }
    return 0;
}

/* Initialize: put null in first byte of name for each slot */
void init_list(void)
{
    register int t;
    for(t=0; t<MAX; ++t) addr_list[t].name[0] = '\0';
}

/* Display menu and return choice */
int menu_select(void)
{
    char s[80]; int c;
    printf("1. Enter a name\n");
    printf("2. Delete a name\n");
    printf("3. List the file\n");
    printf("4. Quit\n");
    do {
        printf("\nEnter your choice: ");
        gets(s); c = atoi(s);
    } while(c<0 || c>4);
    return c;
}

/* Find an unused structure slot; returns -1 if full */
int find_free(void)
{
    register int t;
    for(t=0; addr_list[t].name[0] && t<MAX; ++t) ;
    if(t==MAX) return -1;
    return t;
}

/* Enter a new address */
void enter(void)
{
    int slot; char s[80];
    slot = find_free();
    if(slot==-1) { printf("\nList Full"); return; }
    printf("Enter name: "); gets(addr_list[slot].name);
    printf("Enter street: "); gets(addr_list[slot].street);
    printf("Enter city: "); gets(addr_list[slot].city);
    printf("Enter state: "); gets(addr_list[slot].state);
}

```

```

    printf("Enter zip: ");    gets(s);
    addr_list[slot].zip = strtoul(s, '\0', 10);
}

/* Delete: put null in name[0] of specified slot */
void delete(void)
{
    register int slot; char s[80];
    printf("Enter record #: "); gets(s);
    slot = atoi(s);
    if(slot>=0 && slot<MAX) addr_list[slot].name[0] = '\0';
}

/* List all non-empty entries */
void list(void)
{
    register int t;
    for(t=0; t<MAX; ++t) {
        if(addr_list[t].name[0]) {
            printf("%s\n", addr_list[t].name);
            printf("%s\n", addr_list[t].street);
            printf("%s\n", addr_list[t].city);
            printf("%s\n", addr_list[t].state);
            printf("%lu\n\n", addr_list[t].zip);
        }
    }
    printf("\n\n");
}

```

Key design decisions in the mailing list program:

- An array slot is considered unused if name[0] is the null character '\0'
- find_free() searches for an unused slot and returns its index; returns -1 if all slots are full
- -1 is a safe sentinel because there cannot be a -1 index in an array
- zip is stored as unsigned long int — for international postal codes, a string is often more appropriate

PASSING STRUCTURES TO FUNCTIONS

Passing Structure Members to Functions

When you pass a member of a structure to a function, you are passing the value of that member — it is irrelevant that the value comes from a structure member. Consider this structure:

```

struct fred { char x; int y; float z; char s[10]; } mike;

/* Passing members BY VALUE: */
func(mike.x);      /* passes character value of x */
func2(mike.y);     /* passes integer value of y */
func3(mike.z);     /* passes float value of z */
func4(mike.s);     /* passes ADDRESS of string s (array = pointer) */
func(mike.s[2]);   /* passes character value of s[2] */

```

Passing members BY REFERENCE (using &):

```

/* The & operator precedes the STRUCTURE NAME, not the member name */

```

```
func(&mike.x);      /* passes address of character x */
func2(&mike.y);     /* passes address of integer y */
func3(&mike.z);     /* passes address of float z */
func4(mike.s);     /* passes address of string s — no & needed (s is already a
pointer) */
func(&mike.s[2]);   /* passes address of character s[2] */
```

Passing Entire Structures to Functions

When a structure is used as a function argument, the ENTIRE STRUCTURE is passed using the normal call-by-value method. Any changes made inside the function do NOT affect the structure passed as the argument.

```
#include <stdio.h>
/* Define structure type globally so all functions can see it */
struct struct_type { int a, b; char ch; };
void f1(struct struct_type parm);
int main(void)
{
    struct struct_type arg;
    arg.a = 1000;
    f1(arg);
    return 0;
}
void f1(struct struct_type parm)
{
    printf("%d", parm.a);    /* prints 1000 */
}
```

- The structure type must be declared GLOBALLY (outside main) so all functions can reference it
- Had struct_type been declared inside main(), it would not be visible to f1()

Type Matching Requirement

When passing structures, the type of the argument must match the type of the parameter. It is NOT sufficient for them to be physically similar — their TYPE NAMES must match:

```
/* This program is INCORRECT and will not compile */
struct struct_type { int a, b; char ch; };    /* type 1 */
struct struct_type2 { int a, b; char ch; };   /* type 2 — different name! */

void f1(struct struct_type2 parm);
int main(void)
{
    struct struct_type arg;
    arg.a = 1000;
    f1(arg);    /* TYPE MISMATCH — struct_type != struct_type2 */
    return 0;
}
```

STRUCTURE POINTERS

Declaring a Structure Pointer

Structure pointers are declared by placing ***** in front of the structure variable's name:

```
struct addr *addr_pointer;
/* addr_pointer is a pointer to a structure of type addr */
```

Why Use Structure Pointers?

Reason	Explanation
Performance	Passing a structure by value pushes ALL its members onto the stack. For large structures, this degrades performance significantly. Passing a pointer pushes only ONE address — very fast.
Modify original structure	Passing by value means the function works on a copy. Passing a pointer allows the function to modify the actual structure.
Dynamic data structures	Linked lists, binary trees, and other dynamic structures are built using structure pointers (covered in Part Four).

Using Structure Pointers — The Arrow Operator (->)

To find the address of a structure variable, place **&** before its name. To access members through a pointer, use the **->** (arrow) operator instead of the dot operator:

```
struct bal { float balance; char name[80]; } person;
struct bal *p;
p = &person;          /* p now holds the address of person */

p->balance             /* access the balance member through the pointer */
p->balance = 1234.56; /* assign to balance through the pointer */
```

Situation	Operator to Use	Example
Working directly with structure variable	dot (.)	person.balance
Working through a pointer to a structure	arrow (->)	p->balance

Software timer example — demonstrates structure pointers as function parameters:

```
/* Display a software timer. */
#include <stdio.h>
#define DELAY 128000

struct my_time { int hours; int minutes; int seconds; };
void display(struct my_time *t);
void update(struct my_time *t);
void delay(void);

int main(void)
{
    struct my_time systime;
    systime.hours = 0;
    systime.minutes = 0;
```

```
    systime.seconds = 0;
    for(;;) {
        update(&systime);    /* pass ADDRESS of systime */
        display(&systime);   /* pass ADDRESS of systime */
    }
    return 0;
}

void update(struct my_time *t)
{
    t->seconds++;
    if(t->seconds==60) { t->seconds=0; t->minutes++; }
    if(t->minutes==60) { t->minutes=0; t->hours++; }
    if(t->hours==24)    t->hours = 0;
    delay();
}

void display(struct my_time *t)
{
    printf("%02d:", t->hours);
    printf("%02d:", t->minutes);
    printf("%02d\n", t->seconds);
}

void delay(void)
{
    long int t;
    for(t=1; t<DELAY; ++t) ;    /* adjust DELAY for timing */
}
```

Key points from the timer example:

- `my_time` is declared globally so all functions can see the structure type
- `systime` is a local variable in `main()` — declared as a normal structure variable
- Both `update()` and `display()` receive the ADDRESS of `systime` (using `&systime`)
- `update()` can modify `systime` because it has its address; changes persist after the function returns
- The expression `t->hours==24` reads 'the hours member of the structure pointed to by `t`'

ARRAYS AND STRUCTURES WITHIN STRUCTURES

Array Members

A member of a structure can be an array. The array is accessed using the dot operator followed by array indexing on the member name:

```
struct x {
    int    a[10][10];    /* 10×10 array of ints — an array member */
    float b;
} y;

y.a[3][7]    /* access element at row 3, column 7 of array member a */
```

Nested Structures

When a structure is a member of another structure, it is called a nested structure. Members of nested structures are accessed from outermost to innermost using the dot operator:

```

struct emp {
    struct addr address;    /* nested structure of type addr */
    float    wage;
} worker;

worker.address.zip = 93456;
/* Read: 'the zip field of the address member of the worker object' */
/* Access from outermost (worker) → next level (address) → innermost (zip) */

```

Standard	Maximum Nesting Levels Guaranteed
C89	At least 15 levels of structure nesting
C99	At least 63 levels of structure nesting

UNIONS

A union is a memory location that is shared by two or more different types of variables. A union provides a way of interpreting the same bit pattern in two or more different ways.

General form of a union:

```

union tag {
    type member-name;
    type member-name;
    ...
} union-variables;

```

Example:

```

union u_type {
    int i;
    char ch;
};
union u_type cnvt;    /* declare a variable of type u_type */

/* In cnvt, both i and ch share the same memory location.
   i occupies 2 bytes (16-bit int); ch uses only 1 byte.
   The compiler allocates enough storage for the LARGEST member. */

```

Concept	Explanation
Shared memory	ALL members of a union occupy the same memory address — they overlay each other
Storage allocation	The compiler allocates storage equal to the size of the LARGEST member
Active member	At any given moment, only one member contains a meaningful value
Accessing members	Use dot (.) for direct access; arrow (->) when using a pointer to the union

Accessing Union Members

```

cnvt.i = 10;          /* assign 10 to the integer member i */

/* Accessing through a pointer: */
void func1(union u_type *un)
{
    un->i = 10;        /* assign 10 to i through a pointer */
}

```

putw() Example — Type Conversion Using a Union

Unions are especially useful when nonstandard type conversions are needed. This example uses a union to write a short integer to a disk file one byte at a time — something the standard library cannot do directly:

```

#include <stdio.h>
#include <stdlib.h>

/* Union that overlays a short int with a 2-byte character array */
union pw {
    short int i;
    char      ch[2];
};

int putw(short int num, FILE *fp);

int main(void)
{
    FILE *fp;
    fp = fopen("test.tmp", "wb+");
    if(fp==NULL) { printf("Cannot open file.\n"); exit(1); }
    putw(1025, fp); /* write the value 1025 */
    fclose(fp);
    return 0;
}

int putw(short int num, FILE *fp)
{
    union pw word;
    word.i = num;
    putc(word.ch[0], fp); /* write first byte */
    return putc(word.ch[1], fp); /* write second byte */
}

/* By setting word.i, we can access the same data as ch[0] and ch[1].
   This allows putc() (which writes one byte) to output each byte of the integer. */

```

Unions are also used for:

- Manipulating individual bytes of a floating-point number (e.g., to alter precision or perform unusual rounding)
- Reading hardware registers where the same bits may be interpreted different ways
- Implementing type-safe variant records (tagged unions)

BIT-FIELDS

Unlike some other languages, C has a built-in feature called a bit-field that allows you to access a single bit. Bit-fields can be useful for:

- Storing several Boolean (true/false) variables in one byte when storage is limited

- Analyzing status information encoded into one or more bits within a byte (e.g., hardware device registers)
- Implementing encryption routines that need to access individual bits

General Form of a Bit-Field

A bit-field must be a member of a structure or union. The general form is:

```
type name: length;
/* type = int, signed, or unsigned (C99 also allows _Bool) */
/* name = the field's name */
/* length = number of bits in the field */
```

Serial Port Status Byte Example

A classic use of bit-fields is analyzing hardware registers. Here is a serial port status byte where each bit has a specific meaning:

Bit	Meaning When Set
0	Change in clear-to-send line
1	Change in data-set-ready
2	Trailing edge detected
3	Change in receive line
4	Clear-to-send
5	Data-set-ready
6	Telephone ringing
7	Received signal detected

Representing the status byte using a bit-field structure:

```
struct status_type {
    unsigned delta_cts : 1; /* bit 0: change in clear-to-send */
    unsigned delta_dsr : 1; /* bit 1: change in data-set-ready */
    unsigned tr_edge   : 1; /* bit 2: trailing edge detected */
    unsigned delta_rec : 1; /* bit 3: change in receive line */
    unsigned cts       : 1; /* bit 4: clear-to-send */
    unsigned dsr       : 1; /* bit 5: data-set-ready */
    unsigned ring      : 1; /* bit 6: telephone ringing */
    unsigned rec_line  : 1; /* bit 7: received signal */
} status;

/* Using the bit-field: */
status = get_port_status();
if(status.cts) printf("clear to send");
if(status.dsr) printf("data ready");
status.ring = 0; /* clear the ring field */
```

Unnamed Bit-Fields

You do not have to name each bit-field. Unnamed bit-fields allow you to skip unused bits to reach the one you need:

```
/* Only care about bits 4 (cts) and 5 (dsr) — skip the first 4 bits: */
struct status_type {
    unsigned _ : 4;    /* skip 4 unnamed bits (bits 0-3) */
    unsigned cts: 1;   /* bit 4 */
    unsigned dsr: 1;   /* bit 5 */
} status;
/* Bits after dsr do not need to be specified if not used */
```

Mixing Bit-Fields with Normal Members

Normal structure members can be mixed freely with bit-fields:

```
struct emp {
    struct addr address;    /* normal nested structure member */
    float pay;             /* normal float member */
    unsigned lay_off : 1;   /* 1 bit: laid off or active */
    unsigned hourly : 1;   /* 1 bit: hourly pay or salaried */
    unsigned deductions : 3; /* 3 bits: number of IRS deductions (0-7) */
};
/* Only 1 byte is needed for the three bit-field members.
   Without bit-fields, 3 separate variables would require 3 bytes. */
```

Bit-Field Restrictions

Restriction	Detail
Cannot take address	You cannot use the & operator on a bit-field — bit-fields do not have addressable memory locations
Cannot be arrayed	Bit-fields cannot be declared as arrays
Bit ordering is machine-dependent	You cannot know from machine to machine whether fields run right-to-left or left-to-right — code using bit-fields may have portability issues
Access	Use dot operator for direct access; use arrow operator (->) when accessing through a pointer
Additional restrictions	Various specific implementations may impose other restrictions

ENUMERATIONS

An enumeration is a set of named integer constants. Enumerations are common in everyday life — for example, coins: penny, nickel, dime, quarter, half-dollar, dollar. The keyword `enum` signals the start of an enumeration type.

General form:

```
enum tag { enumeration list } variable_list;
/* Both tag and variable_list are optional — but at least one must be present */
```

```
/* Declare an enumeration type called coin: */
enum coin { penny, nickel, dime, quarter, half_dollar, dollar };

/* Declare a variable of type coin: */
enum coin money;

/* Use enumeration values: */
money = dime;
if(money == quarter) printf("Money is a quarter.\n");
```

Integer Values of Enumeration Symbols

Each symbol in an enumeration stands for an integer value. The first symbol is 0, and each subsequent symbol is one greater than the preceding one:

```
/* Default values for enum coin: */
/* penny=0, nickel=1, dime=2, quarter=3, half_dollar=4, dollar=5 */
printf("%d %d", penny, dime); /* Output: 0 2 */
```

Initializers — Custom Values

You can specify the value of one or more symbols using an initializer (= integer value). Symbols after an initializer receive values incremented from the previous value:

```
enum coin { penny, nickel, dime, quarter=100, half_dollar, dollar };
```

Symbol	Value	How Determined
penny	0	Default (first symbol)
nickel	1	Default (previous + 1)
dime	2	Default (previous + 1)
quarter	100	Explicit initializer
half_dollar	101	Previous (quarter=100) + 1
dollar	102	Previous (half_dollar=101) + 1

I/O of Enumeration Values

One common but erroneous assumption is that enumeration symbols can be input and output directly as strings. They CANNOT — a symbol is simply a name for an integer, not a string:

```
/* This will NOT work: */
money = dollar;
printf("%s", money); /* WRONG — dollar is an integer, not a string */
strcpy(money, "dime"); /* WRONG — cannot copy a string to an enum variable */
```

Method 1 — Use a switch statement to convert to a human-readable string:

```
switch(money) {
```

```

    case penny:      printf("penny");      break;
    case nickel:     printf("nickel");     break;
    case dime:       printf("dime");       break;
    case quarter:    printf("quarter");    break;
    case half_dollar: printf("half_dollar"); break;
    case dollar:     printf("dollar");
}

```

Method 2 — Use an array of strings indexed by the enumeration value (only works when no custom initializers are used):

```

char name[][12] = { "penny", "nickel", "dime", "quarter", "half_dollar", "dollar" };
printf("%s", name[money]); /* uses money's integer value as the array index */
/* Only works if no symbol is initialized, because the array starts at index 0 */

```

NOTE Since enumeration values must be converted manually to human-readable strings for I/O operations, enumerations are most useful in routines that do not require such conversions — such as compiler symbol tables, state machines, and internal logic.

Important Difference Between C and C++

There is a significant difference between C and C++ in how structure, union, and enumeration type names (tags) are used:

Aspect	In C	In C++
Tag type	A structure tag is NOT a complete type name — must use struct keyword to declare objects	A structure tag IS a complete type name — can declare objects without the struct keyword
Declaring an object in C	struct MyStruct obj; (keyword required)	MyStruct obj; (shorter form allowed)
C-style in C++	Perfectly legal and accepted	Legal and accepted — no change needed
Applies to	struct, union, and enum tags	struct, union, and enum tags

NOTE Since C++ accepts C-style declarations, there is no trouble when porting from C to C++. However, if porting C++ code to C, you must add the appropriate struct, union, or enum keyword before each tag name.

USING sizeof TO ENSURE PORTABILITY

Structures and unions can create variables of different sizes, and the actual size may change from machine to machine. The sizeof operator computes the size of any variable or type at compile time and helps eliminate machine-dependent code.

Basic sizeof usage (assuming char=1 byte, int=4 bytes, double=8 bytes):

```
char ch; int i; double f;
printf("%d", sizeof(ch)); /* prints 1 */
printf("%d", sizeof(i)); /* prints 4 */
printf("%d", sizeof(f)); /* prints 8 */
```

sizeof with structures:

```
struct s { char ch; int i; double f; } s_var;

/* sizeof(s_var) is at LEAST 13 (1+4+8), but may be GREATER.
   The compiler may PAD the structure for word or paragraph alignment.
   A paragraph is 16 bytes. Never manually add up member sizes! */

/* Always use sizeof when allocating memory for a structure: */
struct s *p;
p = malloc(sizeof(struct s)); /* correct and portable */
```

sizeof with unions:

```
union u { char ch; int i; double f; } u_var;

/* sizeof(u_var) = 8 (size of the LARGEST member: double)
   A union is always as large as its largest member.
   At run time, it does not matter what u_var is currently holding. */
```

Type	Size Calculation	Key Rule
Primitive (int, char, etc.)	Exact size on the target platform	Use sizeof — do not hardcode bytes
Structure	≥ sum of member sizes (may be larger due to padding)	Always use sizeof; never manually add member sizes
Union	= size of LARGEST member	Always as large as its largest element

NOTE *sizeof is a compile-time operator — all information necessary to compute the size of any variable or type is available at compile time. This is especially useful for portability across different CPU architectures and compilers.*

typedef

You can define new data type names using the keyword typedef. You are NOT creating a new data type — you are defining a new name for an existing type. The new name is IN ADDITION TO, not a replacement for, the existing type name.

General form:

```
typedef type newname;
```

Examples:

```
typedef float balance;
/* Tells the compiler that 'balance' is another name for float */

balance over_due;
/* Declares over_due as a floating-point variable (type balance = float) */

/* typedef names can be further aliased: */
typedef balance overdraft;
/* 'overdraft' is now another name for 'balance', which is another name for 'float' */
```

Use of typedef	Benefit
Portability	If you define your own type names for each machine-dependent data type, only the typedef statements need to change when compiling for a new environment
Self-documentation	Allows descriptive names for standard data types — makes code easier to understand (e.g., balance instead of float)
Simplifying complex declarations	Can hide complex type expressions (e.g., pointers to functions) behind a simple name
Structure type names	Can eliminate the need to type 'struct' every time: typedef struct addr ADDR;

typedef with structures — eliminating the need to use struct keyword each time:

```
typedef struct {
    char name[30];
    char street[40];
    char city[20];
    char state[3];
    unsigned long zip;
} ADDR;

ADDR addr_info;          /* declares a structure variable without 'struct' keyword */
ADDR addr_list[100];     /* declares an array of structures */
ADDR *addr_ptr;          /* declares a pointer to a structure */
```

NOTE Using typedef can make your code easier to read and easier to port. But remember: you are NOT creating a new physical data type — typedef only creates a new name for an existing type.

CHAPTER 7 — QUICK REFERENCE SUMMARY

Structures

Concept	Syntax / Rule
Declaration	struct tag { type member; ... }; — declares the FORM only
Variable creation	struct tag var_name; — creates an instance

Concept	Syntax / Rule
Combined	struct tag { ... } var1, var2; — declares type and variables together
No-tag form	struct { ... } var_name; — unnamed type, only one variable
Member access	var.member (dot operator for direct access)
Pointer member access	ptr->member (arrow operator for pointer access)
Array of structures	struct tag arr[N]; — access: arr[i].member
Nested access	outer.inner.member — from outermost to innermost
Assignment	y = x; — assigns entire structure (must be same type)
Pass to function	Call by value (entire copy); use pointer to avoid copy and allow modification
Type matching	Argument and parameter must have the SAME type name — physical similarity is not enough

Unions, Bit-Fields, Enumerations, typedef

Feature	Key Rules
Union declaration	union tag { type member; ... } var; — all members share same address
Union size	Size = size of LARGEST member
Union member access	Same as structure: dot (.) for direct, arrow (->) for pointer
Bit-field declaration	type name : length; — inside struct or union
Bit-field types	int, signed, unsigned (C99 adds _Bool)
Unnamed bit-field	unsigned : N; — skips N bits without a name
Bit-field restrictions	Cannot take address, cannot be arrayed, bit ordering is machine-dependent
Enumeration declaration	enum tag { sym1, sym2, ... } var; — each symbol is a named integer
Enumeration values	Default: start at 0, increment by 1. Custom: sym=value resets counter.
Enumeration I/O	Symbols are integers — cannot input/output as strings directly; use switch or string array
C vs C++ tags	C requires struct/union/enum keyword before tag; C++ allows tag alone
sizeof rule	Struct: ≥ sum of members (may be padded). Union: = largest member.
typedef form	typedef type newname; — new alias for existing type, not a new type
typedef benefit	Portability and self-documentation; simplifies complex declarations

— End of Chapter 7 Notes —